Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота 6

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

« **Побудування програмної системи з множини об’єктів, керованих повідомленнями** »

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Варіант завдання (Варіант №3)

Розрахунковий номер варіанту: Ж mod 4 = 3 mod 4 = 3.

Програма Lab6 (Менеджер):

1. Користувач вводить значення параметрів **n, Min, Max** у діалоговому вікні.
2. Програма викликає програми **Object2**, **Object3** і забезпечує обмін повідомленнями для передавання та отримання інформації. Автоматично впорядковує вікна на екрані.

Програма Object2 (Генератор):

1. Створює вектор **n** дробових (**double**) чисел у діапазоні **Min – Max**.
2. Показує числові значення у декількох стовпчиках та рядках у власному головному вікні.
3. Записує дані в буфер обміну (Clipboard) у текстовому форматі.

Програма Object3 (Графік):

1. Зчитує дані з буферу обміну.
2. Відображає графік y=f(x) у власному головному вікні, де значення y – це значення вектора, а x – індекси елементів. Графік малюється як математична лінія, що проходить через точки (x, y) у порядку зростання x; осі координат мають підписи числових значень.

Вихідний текст файлів трьох програм

Lab6.cpp

```
#include "framework.h"
#include "Resource.h"
#include <string>

HINSTANCE hInst;
WCHAR szTitle[] = L"Lab6 - Менеджер (Варіант 3)";
WCHAR szWindowClass[] = L"Lab6Class";

int g_n = 10;
double g_Min = 0.0;
double g_Max = 100.0;
```

```
struct ParamsDTO {
    int n;
    double minVal;
    double maxVal;
};

UINT WM_OBJ2_READY = 0;
UINT WM_OBJ3_READY = 0;
UINT WM_OBJ2_DONE  = 0;

ATOM MyRegisterClass(HINSTANCE hInstance);
BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
INT_PTR CALLBACK ParamsDlg(HWND, UINT, WPARAM, LPARAM);

bool StartProcess(LPCWSTR exeName) {
    STARTUPINFOW si;
    PROCESS_INFORMATION pi;
 ZeroMemory(&si, sizeof(si));
  si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    if (CreateProcessW(exeName, NULL, NULL, NULL, FALSE, 0, NULL,
NULL, &si, &pi)) {
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
        return true;
    }
    return false;
}

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE
hPrevInstance, _In_ LPWSTR lpCmdLine, _In_ int nCmdShow) {
    WM_OBJ2_READY = RegisterWindowMessage(L"MSG_OBJ2_READY");
    WM_OBJ3_READY = RegisterWindowMessage(L"MSG_OBJ3_READY");
    WM_OBJ2_DONE  = RegisterWindowMessage(L"MSG_OBJ2_DONE_COPY");

    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;

    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
```

```
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}

ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(nullptr, IDI_APPLICATION);
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB6);
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(nullptr, IDI_APPLICATION);
    return RegisterClassExW(&wcex);
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle,
WS_OVERLAPPEDWINDOW, 50, 50, 400, 200, nullptr, nullptr, hInstance,
nullptr);
    if (!hWnd) return FALSE;
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,
LPARAM lParam) {
    if (message == WM_OBJ2_READY) {
        HWND hObj2 = FindWindowW(L"Object2Class", NULL);
        if (hObj2) {
            ParamsDTO params = { g_n, g_Min, g_Max };
            COPYDATASTRUCT cds;
            cds.dwData = 1;
```

```
            cds.cbData = sizeof(ParamsDTO);
            cds.lpData = &params;
            SendMessage(hObj2, WM_COPYDATA, (WPARAM)hWnd,
(LPARAM)&cds);
        }
        return 0;
    }

    if (message == WM_OBJ2_DONE) {
        HWND hObj3 = FindWindowW(L"Object3Class", NULL);
        if (!hObj3) StartProcess(L"Object3.exe");
        else PostMessage(hObj3,
RegisterWindowMessage(L"MSG_DRAW_GRAPH"), 0, 0);
        return 0;
    }

    if (message == WM_OBJ3_READY) {
        HWND hObj3 = FindWindowW(L"Object3Class", NULL);
        if (hObj3) {
            PostMessage(hObj3,
RegisterWindowMessage(L"MSG_DRAW_GRAPH"), 0, 0);
            HWND hObj2 = FindWindowW(L"Object2Class", NULL);
            if (hObj2) MoveWindow(hObj2, 50, 260, 400, 400, TRUE);
            if (hObj3) MoveWindow(hObj3, 460, 50, 600, 610, TRUE);
        }
        return 0;
    }

    switch (message) {
    case WM_COMMAND: {
        int wmId = LOWORD(wParam);
        switch (wmId) {
        case ID_START_PROCESS:
            if (DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG_PARAMS),
hWnd, ParamsDlg) == IDOK) {
                HWND hObj2 = FindWindowW(L"Object2Class", NULL);
                if (hObj2) PostMessage(hWnd, WM_OBJ2_READY, 0, 0);
                else StartProcess(L"Object2.exe");
            }
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
```

```
            break;
        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
        }
        break;
    }
    case WM_DESTROY: {
        HWND hObj2 = FindWindowW(L"Object2Class", NULL);
        HWND hObj3 = FindWindowW(L"Object3Class", NULL);
        if (hObj2) PostMessage(hObj2, WM_CLOSE, 0, 0);
        if (hObj3) PostMessage(hObj3, WM_CLOSE, 0, 0);
        PostQuitMessage(0);
        break;
    }
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

INT_PTR CALLBACK ParamsDlg(HWND hDlg, UINT message, WPARAM wParam,
LPARAM lParam) {
    switch (message) {
    case WM_INITDIALOG:
        SetDlgItemInt(hDlg, IDC_EDIT_N, g_n, FALSE);
        SetDlgItemTextW(hDlg, IDC_EDIT_MIN,
std::to_wstring(g_Min).c_str());
        SetDlgItemTextW(hDlg, IDC_EDIT_MAX,
std::to_wstring(g_Max).c_str());
        return (INT_PTR)TRUE;
    case WM_COMMAND:
        if (LOWORD(wParam) == IDOK) {
            g_n = GetDlgItemInt(hDlg, IDC_EDIT_N, NULL, FALSE);
            WCHAR buf[256];
            GetDlgItemTextW(hDlg, IDC_EDIT_MIN, buf, 256);
            g_Min = _wtof(buf);
            GetDlgItemTextW(hDlg, IDC_EDIT_MAX, buf, 256);
            g_Max = _wtof(buf);
            EndDialog(hDlg, LOWORD(wParam));
            return (INT_PTR)TRUE;
        } else if (LOWORD(wParam) == IDCANCEL) {
            EndDialog(hDlg, LOWORD(wParam));
```

```
            return (INT_PTR)TRUE;
        }
        break;
    }
    return (INT_PTR)FALSE;
}
```

Object2.cpp

```
#include "framework.h"
#include "Object2.h"
#include <vector>
#include <string>
#include <random>
#include <iomanip>
#include <sstream>

HINSTANCE hInst;
WCHAR szTitle[] = L"Object2 - Генератор даних";
WCHAR szWindowClass[] = L"Object2Class";
std::vector<double> g_data;

UINT WM_OBJ2_READY = 0;
UINT WM_OBJ2_DONE = 0;
struct ParamsDTO {
    int n;
    double minVal;
    double maxVal;
};

ATOM MyRegisterClass(HINSTANCE hInstance);
BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

void CopyToClipboard(HWND hWnd, const std::vector<double>& data) {
    if (!OpenClipboard(hWnd)) return;
   EmptyClipboard();
    std::wstringstream wss;
    for (double val : data) wss << val << L"\n";
    std::wstring textStr = wss.str();
    size_t memSize = (textStr.length() + 1) * sizeof(WCHAR);
    HGLOBAL hMem = GlobalAlloc(GMEM_MOVEABLE, memSize);
    if (hMem) {
```

```
        void* ptr = GlobalLock(hMem);
        memcpy(ptr, textStr.c_str(), memSize);
        GlobalUnlock(hMem);
        SetClipboardData(CF_UNICODETEXT, hMem);
    }
    CloseClipboard();
}

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE
hPrevInstance, _In_ LPWSTR lpCmdLine, _In_ int nCmdShow) {
    WM_OBJ2_READY = RegisterWindowMessage(L"MSG_OBJ2_READY");
    WM_OBJ2_DONE  = RegisterWindowMessage(L"MSG_OBJ2_DONE_COPY");
    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;
    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}

ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(hInstance, MAKEINTRESOURCE(107));
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = NULL;
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(108));
    return RegisterClassExW(&wcex);
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle,
```

```
WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0, 400, 400, nullptr, nullptr,
hInstance, nullptr);
    if (!hWnd) return FALSE;
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,
LPARAM lParam) {
    switch (message) {
    case WM_CREATE: {
        HWND hLab6 = FindWindowW(L"Lab6Class", NULL);
        if (hLab6) PostMessage(hLab6, WM_OBJ2_READY, 0, 0);
        break;
    }
    case WM_COPYDATA: {
        PCOPYDATASTRUCT pcds = (PCOPYDATASTRUCT)lParam;
        if (pcds->dwData == 1) {
            ParamsDTO* params = (ParamsDTO*)pcds->lpData;
            g_data.clear();
            std::random_device rd;
            std::mt19937 gen(rd());
            std::uniform_real_distribution<double> dis(params-
>minVal, params->maxVal);
            for (int i = 0; i < params->n; i++)
g_data.push_back(dis(gen));

            CopyToClipboard(hWnd, g_data);
            InvalidateRect(hWnd, NULL, TRUE);

            HWND hLab6 = FindWindowW(L"Lab6Class", NULL);
            if (hLab6) PostMessage(hLab6, WM_OBJ2_DONE, 0, 0);
        }
        return 1;
    }
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        if (!g_data.empty()) {
            int startX = 10, startY = 10;
            int x = startX, y = startY, columns = 5, colWidth = 70,
```

```
rowHeight = 20, currentItem = 0;
            for (double val : g_data) {
                std::wstringstream wss;
                wss << std::fixed << std::setprecision(2) << val;
                std::wstring text = wss.str();
                TextOutW(hdc, x, y, text.c_str(), text.length());
                currentItem++;
                if (currentItem % columns == 0) { x = startX; y +=
rowHeight; }
                else { x += colWidth; }
            }
        } else {
            std::wstring msg = L"Очікування параметрів від Lab6...";
            TextOutW(hdc, 10, 10, msg.c_str(), msg.length());
        }
        EndPaint(hWnd, &ps);
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

Object3.cpp

```
#include "framework.h"
#include "Object3.h"
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>

HINSTANCE hInst;
WCHAR szTitle[] = L"Object3 - Графік функції";
WCHAR szWindowClass[] = L"Object3Class";
std::vector<double> g_data;

UINT WM_OBJ3_READY = 0;
UINT MSG_DRAW_GRAPH = 0;
```

```
ATOM MyRegisterClass(HINSTANCE hInstance);
BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void ReadFromClipboard(HWND hWnd) {
   if (!OpenClipboard(hWnd)) return;
    if (IsClipboardFormatAvailable(CF_UNICODETEXT)) {
        HGLOBAL hMem = GetClipboardData(CF_UNICODETEXT);
        if (hMem) {
            LPCWSTR ptr = (LPCWSTR)GlobalLock(hMem);
            if (ptr) {
               g_data.clear();
                std::wstringstream wss(ptr);
                double val;
                while (wss >> val) g_data.push_back(val);
                GlobalUnlock(hMem);
            }
        }
    }
    CloseClipboard();
}

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE
hPrevInstance, _In_ LPWSTR lpCmdLine, _In_ int nCmdShow) {
    WM_OBJ3_READY = RegisterWindowMessage(L"MSG_OBJ3_READY");
    MSG_DRAW_GRAPH = RegisterWindowMessage(L"MSG_DRAW_GRAPH");
    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;
    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}

ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
```

```
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(hInstance, MAKEINTRESOURCE(107));
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = NULL;
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(108));
    return RegisterClassExW(&wcex);
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle,
WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0, 600, 600, nullptr, nullptr,
hInstance, nullptr);
    if (!hWnd) return FALSE;
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,
LPARAM lParam) {
    if (message == MSG_DRAW_GRAPH) {
        ReadFromClipboard(hWnd);
        InvalidateRect(hWnd, NULL, TRUE);
        return 0;
    }
    switch (message) {
    case WM_CREATE: {
        HWND hLab6 = FindWindowW(L"Lab6Class", NULL);
        if (hLab6) PostMessage(hLab6, WM_OBJ3_READY, 0, 0);
        break;
    }
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        if (g_data.size() > 1) {
            RECT rect; GetClientRect(hWnd, &rect);
            int padL = 50, padR = 30, padT = 30, padB = 40;
            int drawWidth = rect.right - padL - padR;
```

```
            int drawHeight = rect.bottom - padT - padB;
            double minY = *std::min_element(g_data.begin(),
g_data.end());
            double maxY = *std::max_element(g_data.begin(),
g_data.end());
            if (maxY - minY == 0) { maxY += 1.0; minY -= 1.0; }
            int n = (int)g_data.size();
            double scaleX = (double)drawWidth / (n - 1);
            double scaleY = (double)drawHeight / (maxY - minY);

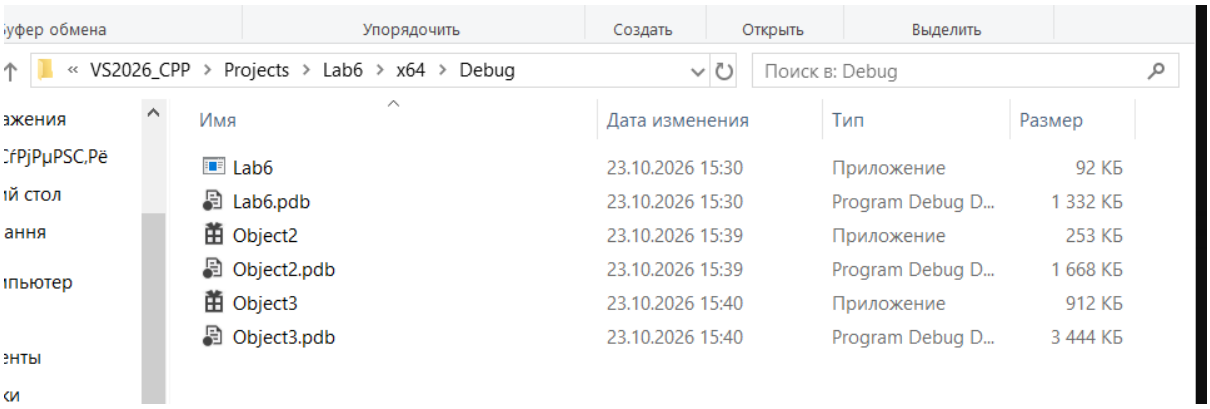
            HPEN hAxisPen = CreatePen(PS_SOLID, 2, RGB(0, 0, 0));
            HPEN hOldPen = (HPEN)SelectObject(hdc, hAxisPen);
            MoveToEx(hdc, padL, padT, NULL); LineTo(hdc, padL, padT
+ drawHeight);
            MoveToEx(hdc, padL, padT + drawHeight, NULL);
LineTo(hdc, padL + drawWidth, padT + drawHeight);

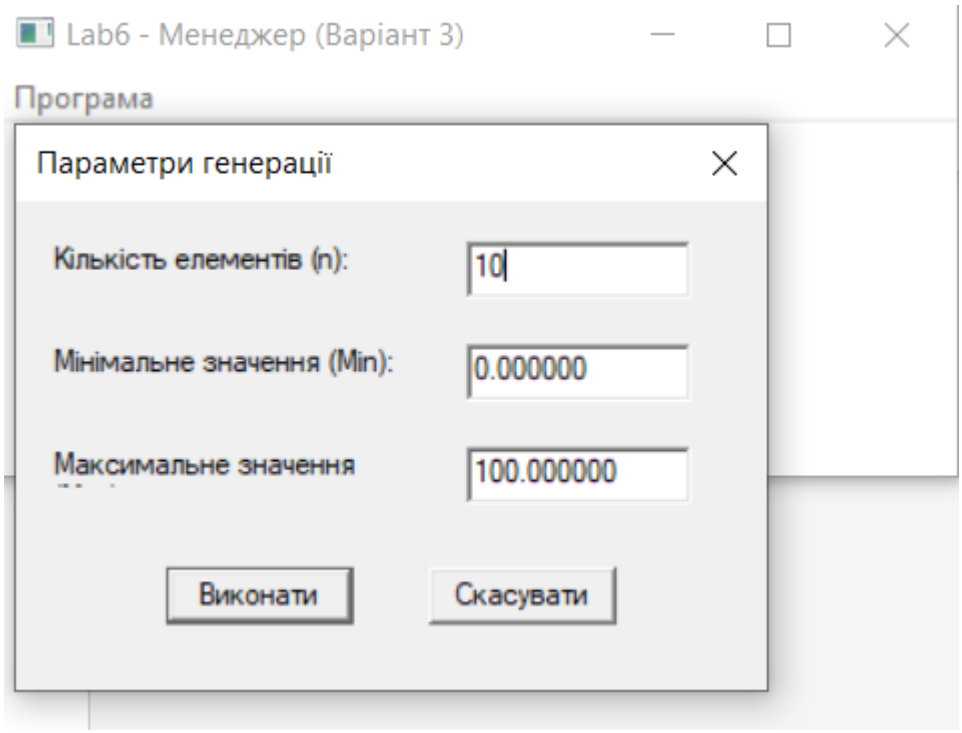
            SetBkMode(hdc, TRANSPARENT); WCHAR buf[32];
            swprintf(buf, 32, L"%.1f", maxY); TextOutW(hdc, 5, padT
- 10, buf, lstrlenW(buf));
            swprintf(buf, 32, L"%.1f", minY); TextOutW(hdc, 5, padT
+ drawHeight - 10, buf, lstrlenW(buf));
            swprintf(buf, 32, L"%d", n - 1);  TextOutW(hdc, padL +
drawWidth - 10, padT + drawHeight + 10, buf, lstrlenW(buf));
            SelectObject(hdc, hOldPen); DeleteObject(hAxisPen);

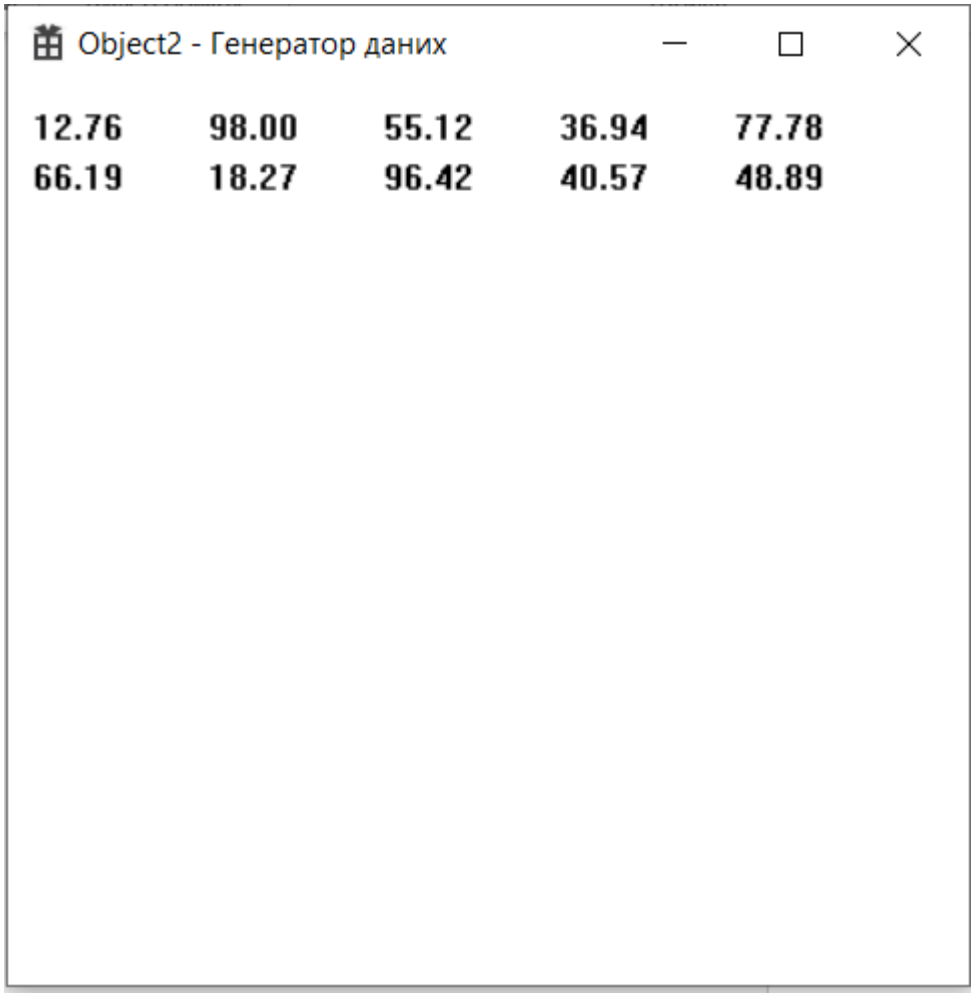
            HPEN hGraphPen = CreatePen(PS_SOLID, 2, RGB(255, 0, 0));
            SelectObject(hdc, hGraphPen);
            for (int i = 0; i < n; i++) {
                int scrX = padL + (int)(i * scaleX);
                int scrY = padT + drawHeight - (int)((g_data[i] -
minY) * scaleY);
                if (i == 0) MoveToEx(hdc, scrX, scrY, NULL);
                else LineTo(hdc, scrX, scrY);
            }
            SelectObject(hdc, hOldPen); DeleteObject(hGraphPen);
        } else {
            std::wstring msg = L"Очікування даних від Object2 (буфер
обміну)...";
            TextOutW(hdc, 20, 20, msg.c_str(), msg.length());
        }
        EndPaint(hWnd, &ps);
```

```
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

Скріншоти


уфер обмена
Упорядочить
Создать
Открыть
Выделить
« VS2026_CPP › Projects › Lab6 › x64 › Debug
Поиск в: Debug
Имя
Дата изменения
Тип
Размер
Lab6
23.10.2026 15:30
Приложение
92 КБ
Lab6.pdb
23.10.2026 15:30
Program Debug D...
1 332 КБ
Object2
23.10.2026 15:39
Приложение
253 КБ
Object2.pdb
23.10.2026 15:39
Program Debug D...
1 668 КБ
Object3
23.10.2026 15:40
Приложение
912 КБ
Object3.pdb
23.10.2026 15:40
Program Debug D...
3 444 КБ


Lab6 - Менеджер (Варіант 3)
Програма
Параметри генерації
Кількість елементів (n):
10
Мінімальне значення (Min):
0.000000
Максимальне значення
100.000000
Виконати
Скасувати


Object2 - Генератор даних
12.76 98.00 55.12 36.94 77.78
66.19 18.27 96.42 40.57 48.89

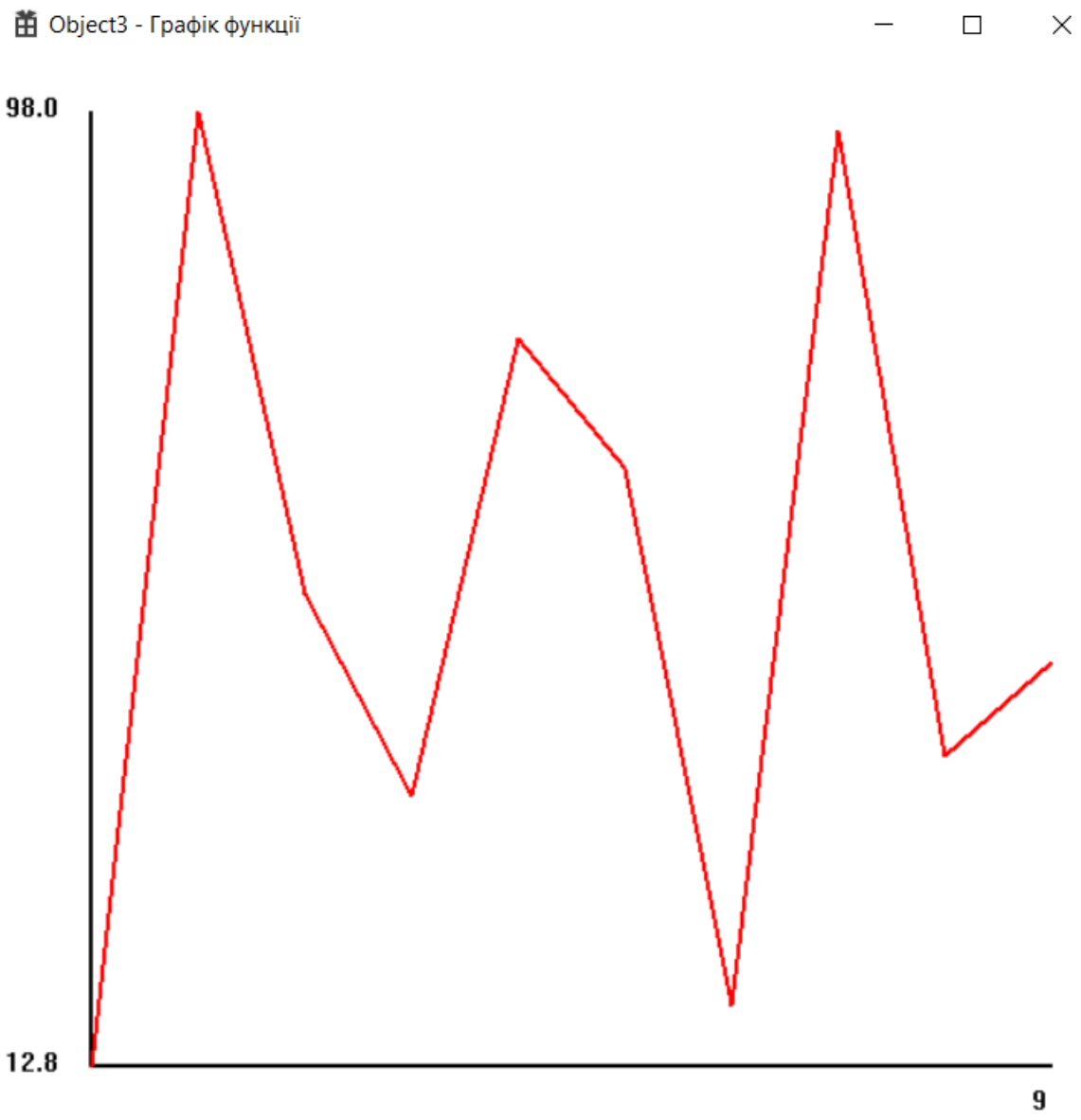
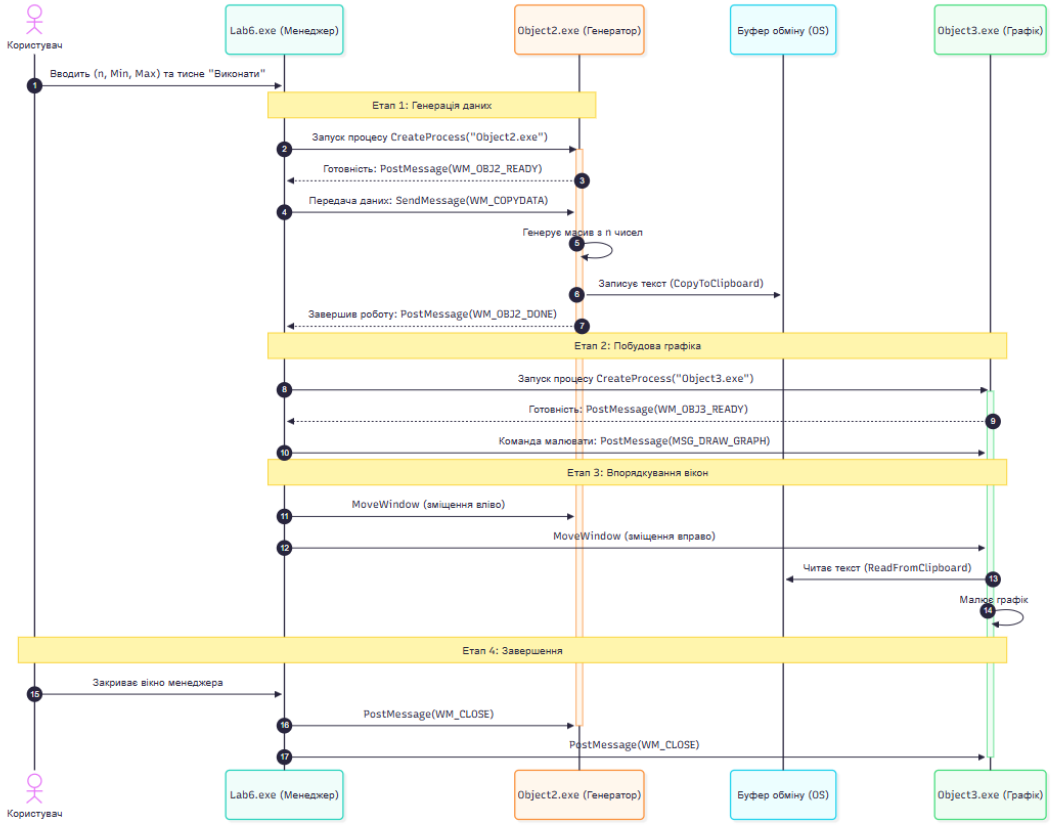

Object3 - Графік функції
98.0
12.8
9

Схема послідовності надсилання-обробки повідомлень



Висновки:

Я отримав вміння та навички використовувати засоби обміну інформацією та запрограмувати взаємодію незалежно працюючих програмних компонентів.